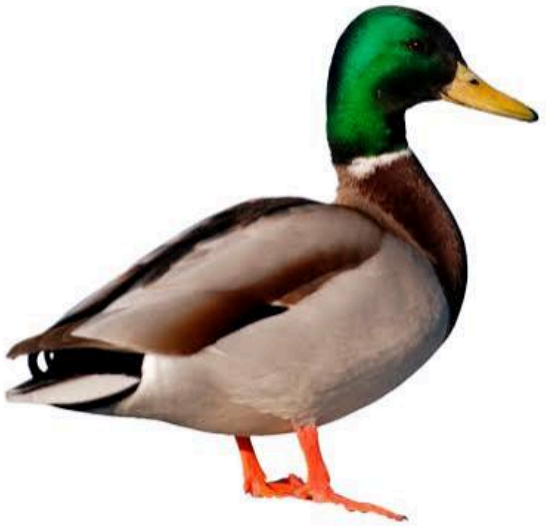
Ciência de Dados Aplicada ao Direito I

Aula 18

Como trabalhar na fase 2



Prof. Bruno da Cunha de Oliveira - 2026/2

Juiz de Python

Insper

Juiz de Python: praticar com correção na hora

Os exercícios da disciplina, corrigidos na hora, em `cdad-juiz.fly.dev`.

O quê	Como funciona
Entrar	com o e-mail do Insper; na primeira vez, chega um código ao e-mail
Questões	as de cada aula aparecem depois que ela é dada
Executar e **Enviar**	Executar roda o exemplo; Enviar roda todos os casos
Gabarito	abre depois do primeiro envio
Diagnóstico	diz por onde começar em cada tema
Nota	nenhuma: é para praticar

Achou um erro numa questão? **Sugerir melhoria**, no alto da página.

Git instalado e conta no GitHub

Insper

Conferir o Git e a conta no GitHub

O que conferir	Está certo quando
No terminal do VS Code, `git --version`	responde `git version` e um número
No navegador, `github.com`	abre com a sua conta, e o seu ícone no alto à direita

Quem ainda não fez: passos 1 e 2 do guia, no Blackboard, agora. No Windows, baixar o instalador do Git e reabrir o VS Code; no Mac, `xcode-select --install`.

Quem usa IA, e para quê

Insper

Como você usou IA nos últimos trinta dias

EXERCÍCIO

Qual destas frases descreve melhor o seu uso de IA nos últimos trinta dias?

A. Quase não usei.

B. Usei de vez em quando, para buscar ou para revisar um texto.

C. Usei toda semana, sempre em conversa: eu pergunto e ela responde.

D. Entreguei a um agente uma tarefa inteira, e ele a construiu sozinho.

O funil do uso de IA

Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.

6 bilhões	quase nenhum contato com modelos de linguagem

O funil do uso de IA

Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.

6 bilhões	quase nenhum contato com modelos de linguagem
1 a 2 bilhões	uso casual de um chatbot gratuito, como uma busca melhor ou uma ajuda para escrever

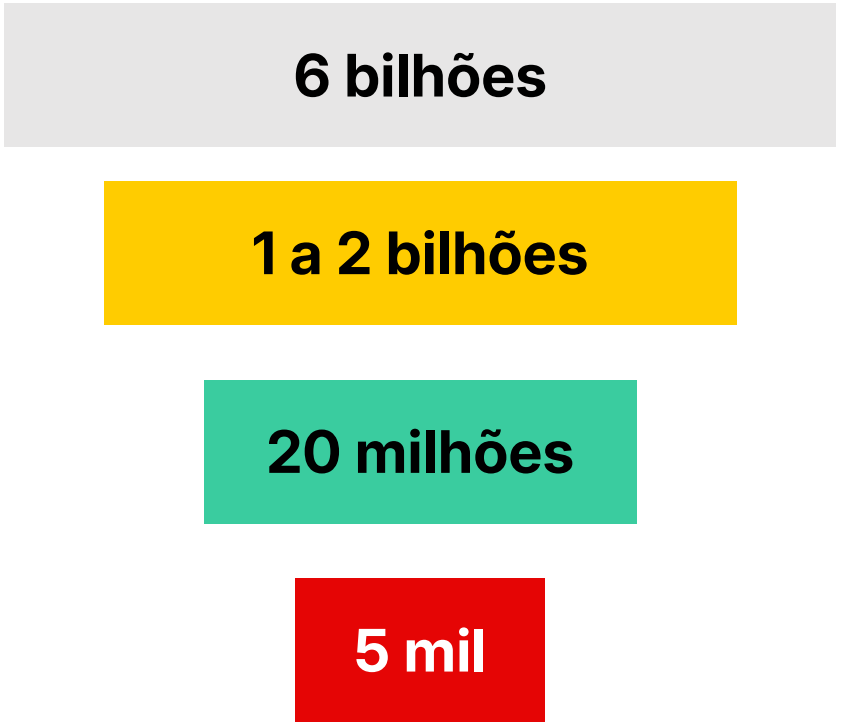
O funil do uso de IA

Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.

6 bilhões	quase nenhum contato com modelos de linguagem
1 a 2 bilhões	uso casual de um chatbot gratuito, como uma busca melhor ou uma ajuda para escrever
20 milhões	uso profissional de modelos de ponta em matemática e código: projetos de semanas feitos por agentes

O funil do uso de IA

Andrej Karpathy, post no X. Resumo; os números são estimativa do próprio autor.



quase nenhum contato com modelos de linguagem

uso casual de um chatbot gratuito, como uma busca melhor ou uma ajuda para escrever

uso profissional de modelos de ponta em matemática e código: projetos de semanas feitos por agentes

acesso a sistemas de ponta de uso interno, ainda não lançados

Fase 2 da APS

Insper

Da fase 1 para a fase 2

Fase 1: vender a ideia

Pitch, site e relatório, para uma banca de advogados.

A IA ajudou vocês a pesquisar, escrever e montar o site.

Da fase 1 para a fase 2

Fase 1: vender a ideia

Pitch, site e relatório, para uma banca de advogados.

A IA ajudou vocês a pesquisar, escrever e montar o site.

Fase 2: construir o produto

O produto funcionando, para uma banca de tecnologia.

É o degrau dos 20 milhões do funil: um assistente de código escreve o programa, e vocês dirigem, pedem, conferem o que ele fez e decidem o que fica.

Mudar de produto, de nome ou de logo

O grupo pode mudar o que apresentou na fase 1.

O que muda	O que fazer
A proposta do produto	Nada: é só desenvolver
O nome da empresa ou do produto	Me avisar antes
A logo	Me avisar antes

Datas da fase 2

Quando	O que acontece
ter 27/10	A Temis apresenta como ganhou a fase 1. Eu explico como a banca avalia e como é a demonstração
ter 17/11	Data limite da entrega do grupo, até o início da aula: endereço do repositório, titular e suplente
qua 18/11	Sessão da fase 2, das 12h às 14h, com banca de pessoas de tecnologia

Para quem investir: contrato e fato relevante por e-mail, a `brunoco2@insper.edu.br`, até o início da aula de 17/11. É separado da entrega do grupo.

De onde partir

Do zero

Vocês descrevem o produto e pedem ao assistente que o construa.

Não há nada para entender antes, e também não há nada funcionando no primeiro dia.

De onde partir

Do zero

Vocês descrevem o produto e pedem ao assistente que o construa.

Não há nada para entender antes, e também não há nada funcionando no primeiro dia.

De um projeto aberto

Vocês copiam para a conta de vocês um projeto público que já funciona.

O trabalho é mudar o que precisa mudar para ele virar o produto de vocês.

Três formas de produto, e as bases de cada uma

Como o advogado usa o produto	Bases
Conversa com um agente que lê, cria e altera arquivos numa pasta	pi
Abre um site, envia um documento ou faz uma pergunta, e vê o resultado na tela	chatbot, da Vercel, ou começar do zero
Não pede nada: o produto trabalha sozinho e avisa	Hermes, NanoClaw ou OpenClaw

Nenhuma é obrigatória, e nenhuma é a recomendada. O README de cada uma diz o que ela faz e como rodar.

Ler um repositório

Leitura em grupo: um repositório da lista

As cinco bases da fase 2 e três bibliotecas de Python que vocês vão usar. Em grupos de três ou quatro, com quem está perto: um repositório por grupo e o formulário "leitura de repositório", no Blackboard, em 12 minutos.

Nome	Endereço, depois de `github.com/`
pi	`earendil-works/pi`
chatbot	`vercel/chatbot`
Hermes	`NousResearch/hermes-agent`
NanoClaw	`nanocoai/nanoclaw`
OpenClaw	`openclaw/openclaw`
pandas	`pandas-dev/pandas`
requests	`psf/requests`
juscraper	`jtrecenti/juscraper`

O que responder sobre o projeto

#	Pergunta	Onde achar
1	O que o projeto faz, do jeito que o grupo entendeu	o `README`, abaixo da lista de arquivos
2	Como se instala e se roda	o `README`
3	Qual é a licença	a barra da direita, em **License**
4	Qual é a linguagem principal	a barra da direita, em **Languages**
5	Quantos commits, e a data do último	acima da lista de arquivos

O que responder sobre a conversa em volta do código

#	Pergunta	Onde achar
6	Quantas issues e quantos pull requests estão abertos	as abas **Issues** e **Pull requests**
7	Uma issue aberta e um pull request aberto, cada um explicado numa frase	abrir um de cada aba
8	O repositório tem um arquivo `AGENTS.md` ou `CLAUDE.md`? O que ele pede?	a lista de arquivos

Vale usar IA para entender o que leram.

Várias pessoas no mesmo código

Ana e Beto no mesmo prazos.py

Os repositórios que vocês leram têm muitas pessoas mexendo no mesmo código. No Google Docs, o grupo inteiro escreve o mesmo relatório ao mesmo tempo. No código, Ana trocou o nome da função para `prazo_dobrado`, e Beto, sem ver, acrescentou a última linha. Cada um testou a sua parte, e funcionava.

```
def prazo_dobrado(dias):
    return dias * 2

print(prazo_dobrado(15))
print(prazo_em_dobro(30))
```

O que aparece quando o programa roda?

Um nome trocado para o programa

RESPOSTA

```
30
NameError: name 'prazo_em_dobro' is not defined. Did you
mean: 'prazo_dobrado'?
```

No Docs, uma frase errada estraga uma frase. No código, um nome trocado para o programa inteiro, e nenhum dos dois errou sozinho.

Um pedido, doze arquivos alterados

Agora é o assistente: alguém do grupo pediu uma mudança, ele alterou doze arquivos, e o que funcionava ontem parou de funcionar.

O que vocês precisariam ter guardado para voltar ao que funcionava?

Versões registradas

RESPOSTA

Uma cópia do projeto em cada ponto em que ele funcionava, com data e descrição, para a qual dê para voltar.

E um jeito de ver o que mudou de uma cópia para a outra.

São as duas coisas que o Git faz.

O histórico do Docs e o do Git

	Google Docs	Git
Quem guarda	o Docs, sozinho	vocês, quando decidem
Nome da versão	data e hora	uma mensagem de vocês
Várias pessoas	no mesmo arquivo, ao mesmo tempo	cada uma na sua cópia
Voltar	**Restaurar esta versão**	a qualquer versão registrada

Git e GitHub

Insper

Git e GitHub

Git

Um programa, no computador de vocês.

Registra as versões de uma pasta e deixa voltar a qualquer uma delas.
A pasta com o histórico se chama repositório.

Git e GitHub

Git

Um programa, no computador de vocês.

Registra as versões de uma pasta e deixa voltar a qualquer uma delas.
A pasta com o histórico se chama repositório.

GitHub

Um site.

Guarda uma cópia do repositório, onde o grupo inteiro e a banca conseguem ver.

O que os grupos acharam

O projeto: linguagem, licença e commits

RESPOSTA

Nome	Linguagem	Licença	Commits	Último commit
pi	TypeScript	MIT	6.996	19/10/2026
chatbot	TypeScript	Apache-2.0	648	08/07/2026
Hermes	Python	MIT	56.681	19/10/2026
NanoClaw	TypeScript	MIT	2.973	19/10/2026
OpenClaw	TypeScript	MIT	114.729	19/10/2026
pandas	Python	BSD-3-Clause	39.552	18/10/2026
requests	Python	Apache-2.0	6.495	21/09/2026
juscraper	HTML	MIT	1.023	14/10/2026

Números de 19/10/2026. Cada commit é uma versão registrada do projeto. O juscraper aparece como HTML porque guarda páginas de tribunais para os testes.

A conversa: issues, pull requests e instruções para o assistente

RESPOSTA

Nome	Issues abertas	PRs abertos	AGENTS.md	CLAUDE.md
pi	264	79	sim	não
chatbot	11	17	não	não
Hermes	16.206	33.784	sim	não
NanoClaw	285	733	sim	sim
OpenClaw	6.253	3.258	sim	não
pandas	1.897	232	sim	não
requests	156	88	não	não
juscraper	53	3	sim	sim

README: o manual do projeto

O `README.md` é a primeira coisa que aparece na página de um repositório. Diz o que o projeto faz, como instalar e como rodar.

O que o grupo não entendeu, a IA ajuda a entender: colem o trecho num chat e peçam a explicação para alguém que nunca programou. Depois confiram no repositório se a explicação bate com o que está lá.

Na fase 2, o README é a porta de entrada do repositório que vocês entregam à banca.

Licença: o que se pode fazer com o código

Licença	O que permite, e o que pede em troca
MIT	usar, mudar, vender; manter o aviso e a licença
Apache 2.0	o mesmo, e dizer o que foi mudado
BSD 3-Clause	o mesmo, sem usar o nome dos autores para promover
GPL e AGPL	quem distribui versão mudada abre o código; na AGPL, também quem a oferece pela internet
proprietária	só quem o dono autorizar usa, mesmo com o código à vista
nenhuma	o mesmo que a proprietária

Issue e pull request

Aba	O que tem nela
Issues	conversas sobre o projeto: uma dúvida, uma sugestão, um defeito encontrado. Não muda arquivo nenhum
Pull requests	propostas de mudança nos arquivos, que o dono lê, comenta e aceita ou recusa. A sigla é PR

AGENTS.md e CLAUDE.md: instruções para o assistente

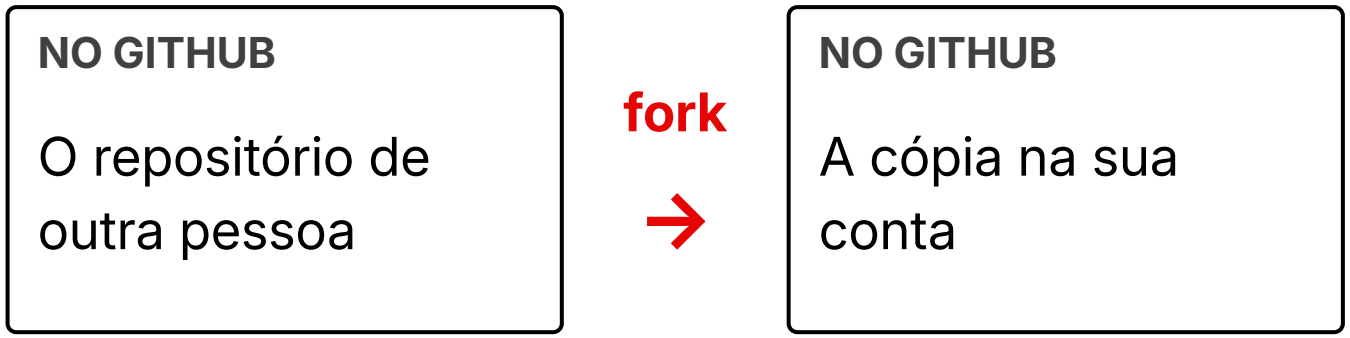
Arquivo	O que é
`AGENTS.md`	um formato aberto de instruções para assistentes de código; o Codex e outros leem
`CLAUDE.md`	as instruções que o Claude Code lê no começo de cada sessão; sem ele, o Claude Code lê o `AGENTS.md`

Na APS: escrevam ali o que o produto faz, como rodar e as regras do grupo, como "responda em português". O assistente de cada um de vocês começa sabendo, sem que ninguém precise repetir.

Uma cópia para cada um

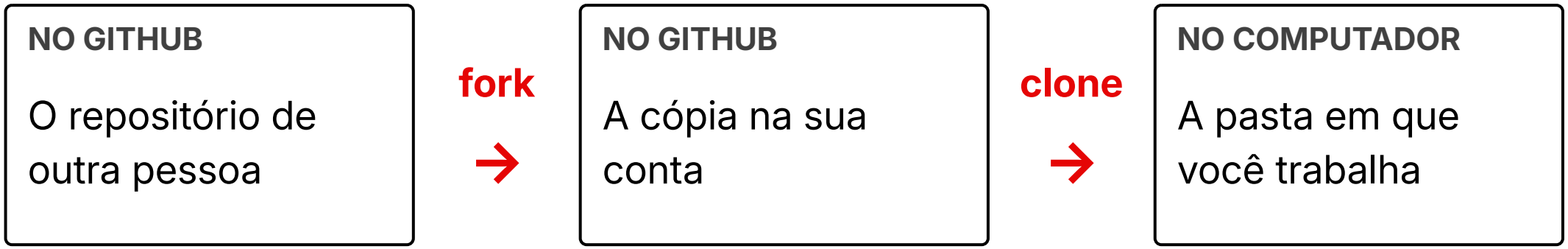
Insper

Fork e clone



Fork é uma cópia, na sua conta do GitHub, de um repositório de outra pessoa. A cópia é sua: você altera à vontade, e o original não muda.

Fork e clone



Fork é uma cópia, na sua conta do GitHub, de um repositório de outra pessoa. A cópia é sua: você altera à vontade, e o original não muda.

Clone é uma cópia, no seu computador, de um repositório que está no GitHub. É nela que você edita os arquivos.

Passo 2: a conta no GitHub

Quem já tem conta entra nela. Quem não tem cria agora:

Ordem	O que fazer
1	Abrir `github.com/signup`
2	Escolher e-mail, senha e nome de usuário: o nome aparece no endereço de tudo o que você publicar
3	Digitar na página o código que o GitHub manda ao seu e-mail

Passo 3: dizer ao Git quem você é

No terminal do VS Code, uma linha de cada vez, com o seu nome e o seu e-mail entre aspas:

```
git config --global user.name "Seu Nome"
git config --global user.email "voce@exemplo.com"
```

As duas linhas não respondem nada. Toda versão registrada leva esse nome e esse e-mail, e num repositório público os dois ficam visíveis.

Passo 4: fazer o fork

Ordem	O que fazer
1	Abrir `github.com/bdcdo/pratica-git`
2	Clicar em **Fork**, no alto à direita, e depois em **Create fork**
3	Na página da sua cópia, clicar em **Code**, na aba **HTTPS**, e copiar o endereço

Deu certo quando o endereço copiado tem o seu nome de usuário depois de `github.com/`.

Passo 4: clonar o fork

No terminal do VS Code, a primeira linha leva à sua pasta de usuário. Na segunda, colem o endereço copiado:

```
cd ~
git clone https://github.com/SEU-USUARIO/pratica-git.git
```

Deu certo quando a resposta começa com `Cloning into 'pratica-git'...` e não traz a palavra `fatal`.

Passo 4: abrir a pasta no VS Code

Ordem	O que fazer
1	Menu **File**, **Open Folder**, e escolher a pasta `pratica-git`
2	Se o VS Code perguntar se você confia nos autores, clicar em **Yes, I trust the authors**
3	Abrir um terminal novo, pelo menu **Terminal**, **New Terminal**

Deu certo quando a lista de arquivos mostra `prazos.py` e `README.md`.

Uma linha a mais em prazos.py

```
def prazo_em_dobro(dias):
    return dias * 2

print(prazo_em_dobro(15))
print(prazo_em_dobro(30))
```

A última linha acabou de ser escrita, e o arquivo foi salvo. O que falta para essa mudança virar uma versão a que dá para voltar, e para ela chegar ao GitHub?

Registrar uma versão e enviar, passo a passo

Animação no navegador: `animacao-git.html`, cenário `registrar`.

status, diff, add, commit e push

RESPOSTA

Comando	O que faz
`git status`	diz o que mudou desde a última versão
`git diff`	mostra a mudança; a tecla q sai
`git add prazos.py`	escolhe um arquivo para a próxima versão
`git add .`	escolhe tudo o que mudou na pasta
`git commit -m "..."`	registra a versão, com uma mensagem
`git push`	envia as versões novas ao GitHub

A mesma versão, no seu fork

Acrescentem ao seu `prazos.py` a linha `print(prazo_em_dobro(30))`, salvem, e rodem no terminal, uma linha de cada vez:

```
git status
git diff
git add .
git commit -m "Mostra o prazo de 30 dias"
git push
```

No primeiro `git push`, o login no GitHub é pelo navegador. Ninguém cria token nem digita senha no terminal.

A versão nova no GitHub

RESPOSTA

A última linha do `git push` termina em `main -> main`.

Na página do seu fork, `prazos.py` mostra a linha nova, e o histórico passa de duas versões para três.

Os códigos das versões de vocês são diferentes dos da animação: cada código depende de quem registrou e de quando.

Se o Git travar

Se aparecer	O que fazer
o terminal pede usuário e senha	não digitar: Ctrl + C, reabrir o VS Code na pasta, confiando nos autores, e tentar de novo
`LF will be replaced by CRLF`	nada: é só aviso
o `git push` responde `GH007`	o conserto está no socorro do guia de Git, ou me chamem

O repositório da fase 2

Insper

Na fase 2, o link é o do repositório

FASE 1: O SITE

`www.grupo-x.com.br`

A banca abriu o site do grupo e leu a proposta.

Na fase 2, o link é o do repositório

FASE 1: O SITE

`www.grupo-x.com.br`

A banca abriu o site do grupo e leu a proposta.

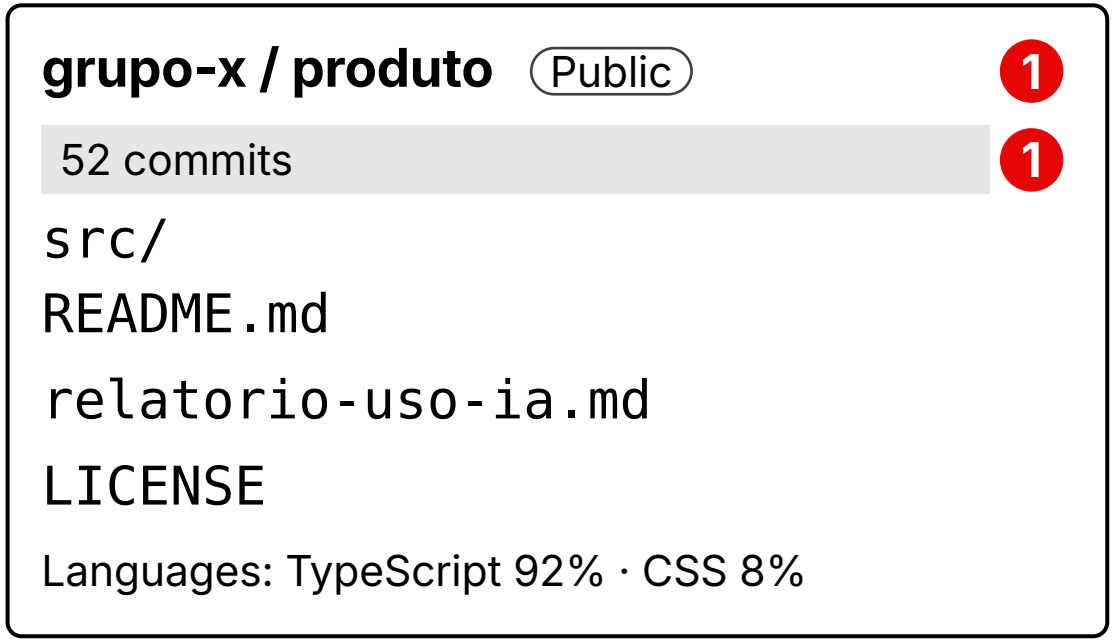
FASE 2: O REPOSITÓRIO

`github.com/grupo-x/produto`

A banca recebe o endereço do repositório, como o do fork que vocês acabaram de enviar: abre o `README` e vê o histórico.

Por isso o repositório precisa estar organizado e fazer sentido para quem nunca o viu.

O que vale para todos os grupos



1. Repositório público, com o histórico de versões

Insper

O que vale para todos os grupos

grupo-x / produto Public 1

52 commits 1

`src/`

`README.md` 2

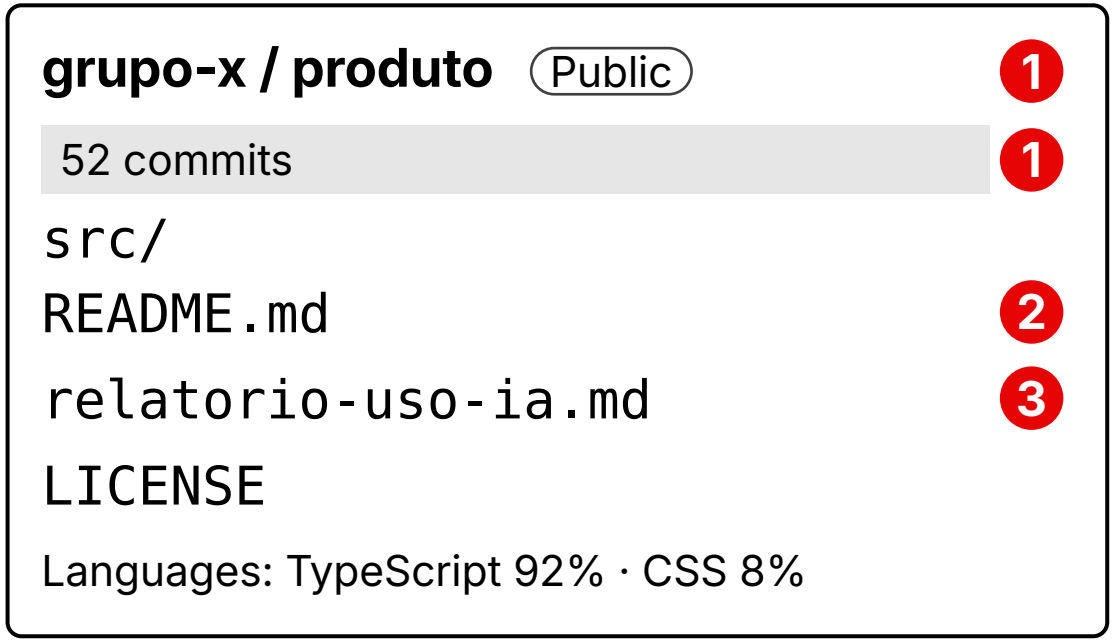
`relatorio-uso-ia.md`

`LICENSE`

Languages: TypeScript 92% · CSS 8%

1. Repositório público, com o histórico de versões
2. `README.md` que diz o que o produto faz e como rodar

O que vale para todos os grupos



1. Repositório público, com o histórico de versões
2. `README.md` que diz o que o produto faz e como rodar
3. Relatório de uso de IA e de outras ferramentas, como na fase 1

O que vale para todos os grupos

grupo-x / produto Public 1

52 commits 1

`src/`

`README.md` 2

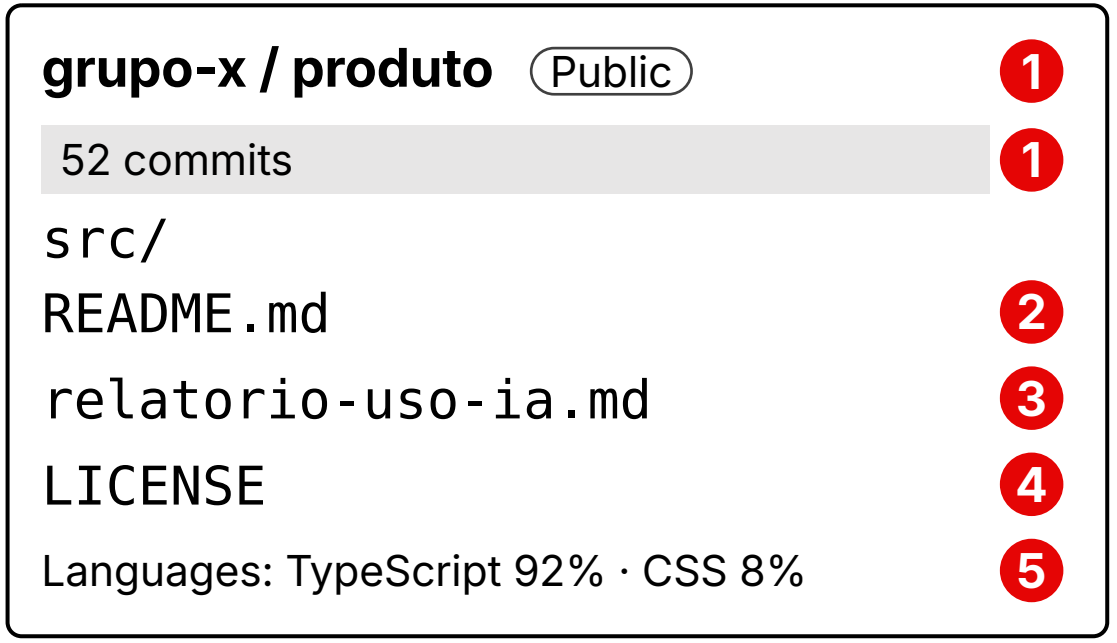
`relatorio-uso-ia.md` 3

`LICENSE` 4

Languages: TypeScript 92% · CSS 8%

1. Repositório público, com o histórico de versões
2. `README.md` que diz o que o produto faz e como rodar
3. Relatório de uso de IA e de outras ferramentas, como na fase 1
4. Uma licença, no arquivo `LICENSE`

O que vale para todos os grupos



1. Repositório público, com o histórico de versões
2. `README.md` que diz o que o produto faz e como rodar
3. Relatório de uso de IA e de outras ferramentas, como na fase 1
4. Uma licença, no arquivo `LICENSE`
5. A linguagem de programação é livre

A licença do grupo: três caminhos

Caminho	Como fica
aberto	seguir com a licença da base, ou escolher uma no GitHub: **Add file**, **Create new file**, o nome `LICENSE` e o botão **Choose a license template**
fechado	licença proprietária: o repositório continua público para a banca, e só quem o grupo autorizar usa; o `LICENSE` diz "Copyright 2026 Grupo X. Todos os direitos reservados."
misto	uma parte aberta e outra fechada

Com uma base MIT, o grupo pode fechar o produto

O CÓDIGO DO GRUPO

a licença que o grupo escolher, inclusive a proprietária

O CÓDIGO QUE VEIO DA BASE

continua MIT, e o aviso de copyright da base fica no repositório

MIT, Apache 2.0 e BSD pedem só que o aviso e a licença da base fiquem junto do código que veio dela.

Com uma base GPL ou AGPL seria diferente: o produto inteiro teria de ficar aberto. Nenhuma das cinco bases é assim.

Misto: o PrairieLearn

O PrairieLearn, em que vocês fazem os notebooks, tem o código inteiro no GitHub, com três licenças:

Parte	Licença	O que permite
edição comunitária	AGPL	usar e mudar; quem oferece pela internet abre o código
edição Enterprise	proprietária	usar em produção só com contrato ou assinatura
código antigo, de Illinois	MIT	usar, mudar e vender, mantendo o aviso

O assistente de código

Insper

Passo 6 do guia: Codex ou Claude

Caminho	Onde roda	Que conta pede
Codex, aplicativo	programa separado	ChatGPT, até a gratuita
Codex, extensão	dentro do VS Code	ChatGPT Edu, Plus ou maior
Claude, aplicativo	à parte, aba **Code**	Claude Pro ou Max
Claude Code, extensão	dentro do VS Code	Claude Pro ou Max

Com o ChatGPT Edu do Insper, o Codex funciona no aplicativo e na extensão. Escolham um, instalem e abram nele a pasta `pratica-git`.

Se o assistente não registrar ou não enviar

Se acontecer	O que fazer
o assistente pede permissão para registrar ou enviar	aprovar
o envio feito pelo assistente falha	rodar `git push` no terminal do VS Code

Um pedido ao assistente

Façam este pedido ao assistente:

> Acrescente ao `prazos.py` a função `dias_restantes(prazo, decorridos)`, que devolve quantos dias faltam. Depois mostre quantos dias restam de um prazo de 15 dias em dobro, passados 12 dias. Registre a versão e envie ao GitHub.

Antes de rodar o programa: que número novo tem que aparecer?

A resposta antes do código

RESPOSTA

```
30
60
18
```

As duas primeiras linhas já estavam no arquivo. A nova é 18: o dobro de 15 é 30, e 30 menos 12 é 18.

Vocês sabem a resposta antes de o assistente escrever uma linha, e é isso que permite conferir o que ele entregou.

Pedir ao assistente que desfaça

A função nova não serve mais. Façam este pedido:

> Desfaça o último commit com um commit novo, sem apagar o histórico, e envie ao GitHub.

Quantas versões o histórico do seu fork passa a ter? A função `dias_restantes` some do histórico?

Desfazer acrescenta uma versão

RESPOSTA

Cinco: as duas que vieram do repositório do curso, a de vocês, a do assistente e a que desfaz.

A função continua no histórico, na quarta versão. Desfazer não apaga nada: registra uma versão nova com o conteúdo de antes.

Entreguem o endereço do seu fork pelo formulário "fork da aula 18", no Blackboard.

Duas atividades no GitHub

Insper

Abrir o repositório original

O seu fork tem cinco versões. Abram `github.com/bdcdo/pratica-git`, o repositório de onde ele saiu.

Quantas versões o original tem?

O original não muda, e o pull request propõe

RESPOSTA

Duas: as mesmas de antes do fork, sem nenhuma linha de vocês.

O fork é uma cópia. O que vocês registram e enviam fica nele, e o original só muda se o dono aceitar.

Para propor ao dono uma mudança feita no fork, existe o pull request, que vocês viram aberto aos milhares nos repositórios da leitura.

Do fork ao pull request

NO SEU FORK

A versão nova, registrada e enviada

pull request →

NO ORIGINAL

O dono lê, comenta e aceita ou recusa

O pull request mostra ao dono o que mudou e deixa uma conversa aberta sobre a mudança.

Do fork ao pull request

NO SEU FORK

A versão nova, registrada e enviada

pull request →

NO ORIGINAL

O dono lê, comenta e aceita ou recusa

O pull request mostra ao dono o que mudou e deixa uma conversa aberta sobre a mudança.

Aceito, a versão entra no original. Recusado, ela continua só no fork de quem propôs.

Issue: ajudar um projeto sem programar

A issue, que vocês também viram na leitura, não muda arquivo nenhum, e é uma das formas mais úteis de ajudar um projeto:

Para	Como escrever
tirar uma dúvida	o que vocês tentaram fazer e onde travaram
dar uma sugestão	o que falta e por que faria diferença para quem usa
apontar um defeito	o que fizeram, o que esperavam e o que aconteceu

Pull request feito com IA, no repositório dos outros

Antes de abrir um pull request no projeto de outra pessoa, leiam o `CONTRIBUTING.md` e o `AGENTS.md`. Alguns projetos não aceitam código gerado por IA, e outros pedem que se avise.

NetBSD, diretrizes de commit. Resumo.
Código gerado por IA é presumido contaminado, e só entra com autorização escrita da equipe principal.

Atividade 1: uma issue e um pull request

#	No repositório `bdcdo/praticas-github-cdad-2026-2`
1	Abrir uma issue explicando o que é uma issue, como você explicaria a um advogado que nunca programou
2	Fork, clone e, no VS Code, o arquivo `explicacoes/seu-usuario.md`, explicando o que é um pull request ao mesmo advogado
3	Registrar e enviar com `git add`, `git commit` e `git push`, e abrir o pull request a partir do fork

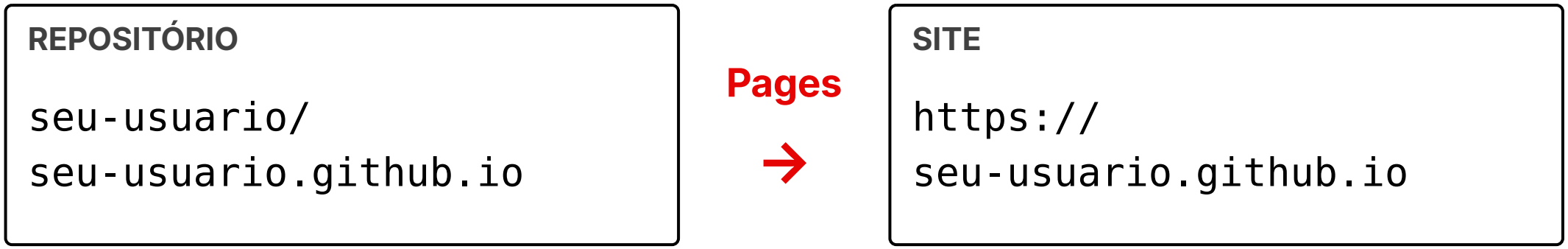
Até o início da aula de terça, 27/10, pelo formulário "issue e PR", no Blackboard. Escritos por você, sem assistente; o passo a passo está no guia.

Quanto vale a atividade 1

Quem	Ponto extra
Abriu a issue e o pull request	0,1
A melhor explicação de issue	mais 0,1
A melhor explicação de pull request	mais 0,1

O ponto extra vai para a prova final. A mesma pessoa pode ganhar os dois prêmios e chegar a 0,3. No empate, ganha quem enviou primeiro.

GitHub Pages: um repositório que vira site



O GitHub Pages publica os arquivos de um repositório como site. Um repositório com o nome exato `seu-usuario.github.io`, em minúsculas, abre nesse endereço; o `.io` é o domínio do Território Britânico do Oceano Índico. Na conta gratuita, ele precisa ser público.

Atividade 2: o seu site pessoal

#	O site tem
1	Endereço `seu-usuario.github.io`, abrindo para qualquer pessoa
2	Uma apresentação sua
3	Link para o seu currículo Lattes
4	Link para o seu perfil no LinkedIn

Quanto vale a atividade 2

Link	Está atualizado quando
Lattes	a "Última atualização" é de 2026
LinkedIn	o perfil tem foto, título e a formação no Insper

Vale 0,1 ponto extra na prova final para quem tiver os quatro itens no site e os dois links atualizados até o início da aula de quarta, 25/11, e mandar o endereço por e-mail a `brunoco2@insper.edu.br`. O passo a passo está no guia.

Na próxima aula

Como um programa entra num site, pede uma informação e traz a resposta de volta, com o `requests` e o juscraper.

Até o início da aula de terça, 27/10: a issue e o pull request da atividade 1.